

✓ TUGAS WEEK 3

Web Scraping

Kelompok 5; NIM ADEL (073) & (160)

Scraping With HTTPS

```
import requests
```

```
page = requests.get('https://github.com/')  
page
```

```
# get the html document  
cont = page.content.decode('utf-8')  
print(cont)
```

HTML Parser using BeautifulSoup

```
from bs4 import BeautifulSoup
```

```
# get the html document  
import requests  
page = requests.get('https://github.com/')  
bs = BeautifulSoup(page.text, 'html.parser')  
bs
```

Tagging in HTML

```
# get title of pages  
bs.title
```

```
bs.title.name
```

```
bs.title.string
```

```
bs.div
```

```
bs.span
```

```
bs.span.text
```

Find and Find_all

```
bs.find("span")
```

```
bs.find_all("span")
```

```
[(div.get('class'), div.get_text(strip=True)[:50]) for div in bs.find_all('div')[:20]]
```

```
data = bs.find('div')  
print(data)  
  
data = bs.find_all('div')  
  
for i in data[:10]:  
    print(i)
```

```
for i in data[:10]:
    print(i.get('class'))
```

```
data = bs.find_all('div')

for i in data[:10]:
    print(i.find('a'))
```

```
root = 'https://github.com/topics/python?page='

repos = []
links = []
descriptions = []

for page in range(1, 4):
    url = requests.get(root + str(page))
    soup = BeautifulSoup(url.text, 'html.parser')
    scrap = soup.find_all('article')

    for i in scrap:
        repos.append(i.find('h3'))
        links.append(i.find('a'))
        descriptions.append(i.find('p'))

print(repos)
print(links)
print(descriptions)
```

```
len(repos), len(links), len(descriptions)
```

```
from numpy import take_along_axis
df = pd.DataFrame({
    'Repos': repos,
    'Links': links,
    'Descriptions': descriptions
})

df
```

Scraping With API

```
url = 'https://suggestqueries-clients6.youtube.com/complete/search?ds=yt&hl=id&gl=id&client=youtube&gs_ri=youtube&sugexp=uqapst'
api_request = requests.get(url)
api_request
```

```
import requests
import json
api_key = 'AIzaSyCdU0eaAlUiVLxJEcKkRHuJwhBuqEZj-OM'
```

```
query = 'web scraping'
max_result = '10'

url = f"https://www.googleapis.com/youtube/v3/search?part=snippet&maxResults={max_result}&q={query}&key={api_key}"

response = requests.get(url)
data = response.json()
data
```

```
for item in data["items"]:
    print(item["snippet"]["title"], "-", item["id"].get("videoId"))
```

```
video_id = '1PCWwK0AsE0'
max_result = '15'

url = f"https://www.googleapis.com/youtube/v3/commentThreads?part=snippet&videoId={video_id}&maxResults={max_result}&key={api_k"

response = requests.get(url)
data = response.json()
```

```
with open("response.json", "w") as f:
    json.dump(data, f, indent=4)
```

```
with open("response.json", "r") as f:
    data = json.load(f)
```

```
with open("response.json", "r") as f:
    data = json.load(f)

authors = []
comments = []

for item in data["items"]:
    top_comment = item["snippet"]["topLevelComment"]["snippet"]
    author = top_comment["authorDisplayName"]
    text = top_comment["textDisplay"]
    authors.append(author)
    comments.append(text)

import pandas as pd
df = pd.DataFrame({'Authors':authors, 'Comments':comments})
df
```

▼ TUGAS WEEK 4

TEXT PROCESSING

```
import warnings
warnings.filterwarnings(action='ignore')
```

```
import requests
from bs4 import BeautifulSoup

page = requests.get('https://github.com/')
bs = BeautifulSoup(page.text, 'html.parser')

bs
```

```
text = bs.get_text(separator=' ', strip=True)

print(text[:2000])
```

GitHub · Change is constant. GitHub keeps you ahead. · GitHub Skip to content Navigation Menu Sign in Platform AI CODE CREATION

```
import re
```

```
#lowercase
hasil_cf = str.lower(text)
print(hasil_cf)
```

```
#remove punctuation
import string
punc = string.punctuation
def remove_punctuation(text):
    return text.translate(str.maketrans('', '', punc))

hasil_rp = remove_punctuation(text)
print(hasil_rp)
```

```
#stopwords
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords')

print(stopwords.words('english'))
print(stopwords.words('indonesian'))
```

```
sw = set(stopwords.words('english'))
def remove_stopwords(text):

    return " ".join([word for word in str(text).split() if word not in sw])

hasil_st = remove_stopwords(text)
print(hasil_st)
```

```
# Stemmer

import nltk
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

print(stemmer.stem('developers'))    # => develop
print(stemmer.stem('developing'))    # => develop
print(stemmer.stem('applications'))  # => applic
print(stemmer.stem('connected'))     # => connect

sentence = "The future of building happens together"

print(" ".join(
    stemmer.stem(word)
    for word in sentence.split()
))
```

```
develop
develop
applic
connect
the futur of build happen togeth
```

```
from nltk.stem import LancasterStemmer
ls = LancasterStemmer()
ls.stem('developers'), ls.stem('developing'), ls.stem('applications'), ls.stem('connected')
```

```
('develop', 'develop', 'apply', 'connect')
```

```
from nltk.stem import SnowballStemmer
ss = SnowballStemmer("english")
ss.stem('student'), ss.stem('studying'), ss.stem('studies'), ss.stem('learning')
```

```
('student', 'studi', 'studi', 'learn')
```

```
import nltk
from nltk.stem import WordNetLemmatizer

nltk.download('wordnet')
wnl = WordNetLemmatizer()

# Lemmatize nouns
print(wnl.lemmatize('developers', 'n'))    # developer
print(wnl.lemmatize('applications', 'n'))  # application
# Lemmatize verbs
print(wnl.lemmatize('building', 'v'))      # build
print(wnl.lemmatize('happens', 'v'))       # happen
print(wnl.lemmatize('connected', 'v'))     # connect
```

```
developer
application
build
happen
connect
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
```

```
sentence = 'I am a Data Science student studying at Surabaya State University'
nltk.download('punkt_tab')
nltk.download('wordnet')
tokens = nltk.word_tokenize(sentence)
print(tokens)
```

```
['I', 'am', 'a', 'Data', 'Science', 'student', 'studying', 'at', 'Surabaya', 'State', 'University']
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...  
[nltk_data] Package wordnet is already up-to-date!
```

```
import nltk  
  
nltk.download('punkt_tab')  
nltk.download('averaged_perceptron_tagger_eng')  
nltk.download('universal_tagset')  
  
sentence = 'I am a Data Science student studying at Surabaya State University'  
  
tokens = nltk.word_tokenize(sentence)  
  
tagged_sent = nltk.pos_tag(tokens, tagset='universal')  
  
print(tagged_sent)
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...  
[nltk_data] Package punkt_tab is already up-to-date!  
[nltk_data] Downloading package averaged_perceptron_tagger_eng to  
[nltk_data] /root/nltk_data...  
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.  
[nltk_data] Downloading package universal_tagset to /root/nltk_data...  
[nltk_data] Downloading package universal_tagset to /root/nltk_data...  
[nltk_data] Unzipping taggers/universal_tagset.zip.  
[nltk_data] Unzipping taggers/universal_tagset.zip.
```